6. Python Program to Implement List Methods (Add, Append, Extend & Delete)

Aim

To write a Python program to implement list methods such as Add, Append, Extend, and Delete.

Algorithm

1. Start the program.
2. Create a list with some elements.
3. Use append() to add an element at the end of the list.
4. Use extend() to add multiple elements to the list.
5. Use insert() to add an element at a specific position (Add).
6. Use del to delete an element from the list.
7. Display the list after each operation.
8. Stop the program.

Python Code

```
# Program to implement list methods

# Create a list
my_list = [10, 20, 30]
print("Original List:", my_list)

# Add an element at a specific position
my_list.insert(1, 15)
print("After Add:", my_list)

# Append an element at the end
my_list.append(40)
print("After Append:", my_list)

# Extend the list with multiple elements
my_list.extend([50, 60])
```

```
print("After Extend:", my_list)


# Delete an element
del my_list[2]
print("After Delete:", my_list)
```

output

```
    # Delete an element
    del my_list[2]
    print("After Delete:", my_list)

...  Original List: [10, 20, 30]
    After Add: [10, 15, 20, 30]
    After Append: [10, 15, 20, 30, 40]
    After Extend: [10, 15, 20, 30, 40, 50, 60]
    After Delete: [10, 15, 30, 40, 50, 60]
```

7. Python Program to Illustrate Different Set Operations

Aim

To write a Python program to perform different set operations such as union, intersection, difference, and symmetric difference.

Algorithm

1. Start the program.
2. Create two sets.
3. Perform union, intersection, difference, and symmetric difference.
4. Display the results.
5. Stop the program.

Code

```
# Program to illustrate set operations


A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

```
print("Set A:", A)
print("Set B:", B)

print("Union:", A | B)
print("Intersection:", A & B)
print("Difference (A-B):", A - B)
print("Symmetric Difference:", A ^ B)
```

output

```
print("Difference (A-B):", A - B)
print("Symmetric Difference:", A ^ B)
```

```
Set A: {1, 2, 3, 4}
Set B: {3, 4, 5, 6}
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference (A-B): {1, 2}
Symmetric Difference: {1, 2, 5, 6}
```

8. Python Program to Generate Calendar for the Given Month and Year

Aim

To write a Python program to display the calendar for a given month and year.

Algorithm

1. Start the program.
2. Import the calendar module.
3. Get the month and year from the user.
4. Display the calendar using calendar.month().
5. Stop the program.

Code

```
# Program to generate a calendar

import calendar

year = int(input("Enter year: "))
```

month = int(input("Enter month (1-12): "))

print(calendar.month(year, month))

output

```
print(calendar.month(year, month))
```

```
Enter year: 2007
Enter month (1-12): 6
     June 2007
Mo Tu We Th Fr Sa Su
                1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30
```

9. Python Program to Remove Punctuations from the Given String

AIM

To write a Python program to remove punctuation marks from a given string.

ALGORITHM

1. Start the program.
2. Import the string module.
3. Get a string from the user.
4. Create an empty string to store the result.
5. Remove all punctuation marks.
6. Display the resulting string.
7. Stop the program.

CODE

Program to remove punctuations

import string

text = input("Enter a string: ")

result = ""

```
for ch in text:
    if ch not in string.punctuation:
        result += ch

print("String without punctuations:", result)
```

OUTPUT

```
for ch in text:
    if ch not in string.punctuation:
        result += ch

print("String without punctuations:", result)
```

```
Enter a string: Hello, World! How are you?
String without punctuations: Hello World How are you
```

10. Python Program to Solve the 8-Puzzle Problem

Aim

To write a Python program to solve the 8-puzzle problem using the A search algorithm*.

Algorithm

1. Start the program.
2. Define the initial state and goal state.
3. Calculate the Manhattan distance heuristic.
4. Use the A* algorithm to find the best possible moves.
5. Move the blank tile in valid directions.
6. Continue until the goal state is reached.
7. Display the solution steps.
8. Stop the program.

Code

```
# 8-Puzzle Problem using A* Search

import heapq
```

```
start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

def heuristic(state):
    distance = 0
    for i, value in enumerate(state):
        if value != 0:
            target = goal.index(value)
            distance += abs(i // 3 - target // 3)
            distance += abs(i % 3 - target % 3)
    return distance

def get_neighbors(state):
    zero = state.index(0)
    row, col = divmod(zero, 3)
    neighbors = []

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        r, c = row + dr, col + dc
        if 0 <= r < 3 and 0 <= c < 3:
            pos = r * 3 + c
            new_state = list(state)
            new_state[zero], new_state[pos] = (
```

```
            new_state[pos], new_state[zero]
        )
        neighbors.append(tuple(new_state))

    return neighbors

def solve(start):
    queue = [(heuristic(start), 0, start)]
    parent = {start: None}
    cost = {start: 0}

    while queue:
        _, g, state = heapq.heappop(queue)

        if state == goal:
            path = []
            while state is not None:
                path.append(state)
                state = parent[state]
            return path[::-1]

        if g != cost[state]:
            continue

        for neighbor in get_neighbors(state):
            new_cost = g + 1

            if neighbor not in cost or new_cost < cost[neighbor]:
                cost[neighbor] = new_cost
                parent[neighbor] = state
```

```
            priority = new_cost + heuristic(neighbor)
            heapq.heappush(
                queue, (priority, new_cost, neighbor)
            )

    return None

solution = solve(start)

if solution:
    print("Solution found:")
    for step, state in enumerate(solution):
        print("\nStep", step)
        for i in range(0, 9, 3):
            print(state[i:i+3])
else:
    print("No solution found.")
```

OUTPUT

```
            for i in range(0, 9, 3):
                print(state[i:i+3])
    else:
        print("No solution found.")
```

```
Solution found:

Step 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 2
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```